

VARIANZA DELLA LOSS FUNCTION NEI NPQC

Introduzione

I Variational Quantum Algorithms (VQA) sono un approccio computazionale moderno ed estremamente efficiente per risolvere molte tipologie di problemi. Tuttavia, al crescere della dimensione del circuito si incontra una concentrazione esponenziale della funzione di loss attorno alle soluzioni, fenomeno noto come Barren Plateau (BP), che rende questi circuiti inutilizzabili.

E' stata studiata la presenza di BP in circuiti con rumore unitario tramite formalismo di Lie, tuttavia, i risultati ottenuti non sono generalizzabili al caso in cui rumore sia generico. L'articolo da cui siamo partiti propone di utilizzare un nuovo formalismo, fatto da matrici non negative, per descrivere l'andamento della varianza nel caso di un rumore-entangling generico.

Il lavoro di questo Notebook vuole spiegare intuitivamente l'articolo ed esporre dei risultati numerici ottenuti sulla falsa riga dello studio seguente.

```
In [3]: from IPython.display import Image, display

image = "Img/Screenshot 2025-02-22 alle 12.50.24.png"

display(Image(filename=image, width=600, height=400))
```

Estimates of loss function concentration in noisy parametrized quantum circuits

Giulio Crognaletti,^{1,2,3,*} Michele Grossi,^{3,†} and Angelo Bassi^{1,2}

¹*Department of Physics, University of Trieste, Strada Costiera 11, 34151 Trieste, Italy*
²*Istituto Nazionale di Fisica Nucleare, Trieste Section, Via Valerio 2, 34127 Trieste, Italy*

³*European Organization for Nuclear Research (CERN), 1211 Geneva, Switzerland*

(Dated: October 4, 2024)

Circuiti quantistici parametrizzati (PQC)

I circuiti quantistici parametrizzati (PQC) sono utilizzati in algoritmi ibridi quantistici-classici per risolvere problemi computazionali, infatti sono la base per gli Algoritmi Quantistici Variazionali (VQA).

Le componenti principali dei VQA includono porte quantistiche parametrizzate ed un ottimizzatore classico, che regola i parametri per minimizzare o massimizzare una funzione obiettivo, generalmente il valore di aspettazione di un osservabile.

Gli algoritmi VQA funzionano in modo iterativo, alternando elaborazione quantistica e classica, e la scelta dell'ansatz di partenza influisce

sull'ottimizzazione.

Le funzioni di costo traducono i parametri quantistici in valori reali, facendo sì che l'ottimizzatore cerchi i minimi su una ipersuperficie definita da:

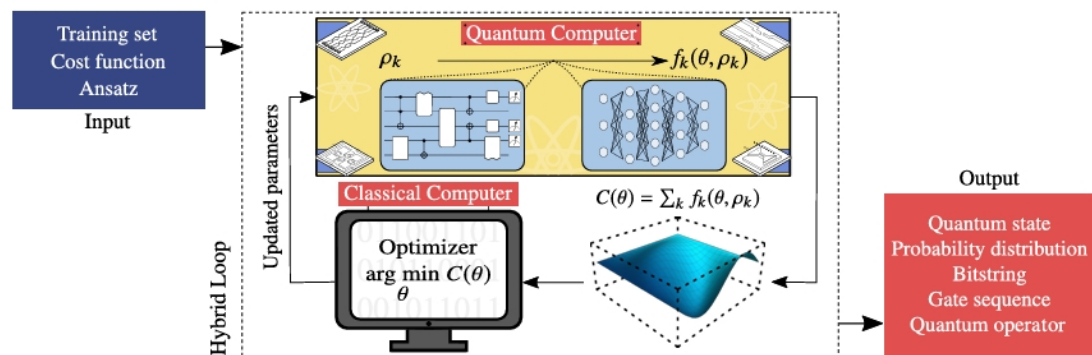
$$C(\theta) = \sum_k f_k(\text{Tr}(O_k \rho_k(\theta)))$$

```
In [4]: from IPython.display import Image, display

image5 = "Img/Screenshot_23-1-2025_104642_.jpeg"

display(Image(filename=image5, width=600, height=400))
```

2



Barren Plateau

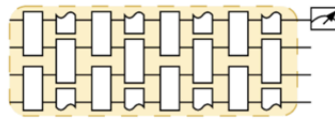
Il fenomeno del *Barren Plateau* è un fenomeno tipico dei circuiti quantistici parametrizzati (PQC), dove il gradiente della funzione di costo tende ad annullarsi esponenzialmente all'aumentare del numero di qubit: in pratica, la funzione è piatta su tutto lo spazio. Questo effetto rende difficile l'addestramento degli algoritmi quantistici variazionali, poiché la probabilità che il gradiente lungo una direzione significativa sia diverso da zero diminuisce rapidamente con l'aumentare dei qubit, rendendo molto complessa la scelta della direzione per le iterazioni successive.

Questo fenomeno emerge sia dalla dimensionalità del sistema che dall'entanglement del circuito iniziale e dell'osservabile.

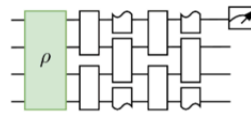
```
In [5]: image1 = "Img/Screenshot 2025-01-23 alle 11.32.53.png"
display(Image(filename=image1, width=500, height=400))
```

Trainability – Theoretical Limitations

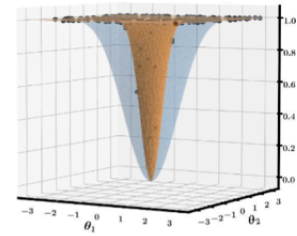
Exponential Decay in the loss function $\text{Var}[\ell_{\theta}(\rho, O)] \in \mathcal{O}\left(\frac{1}{b^n}\right)$



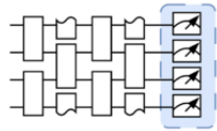
Circuit Expressivity



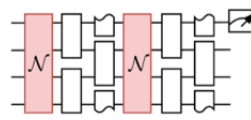
Entangled Initial State



Probabilistic
Barren plateau



Measurement Locality



Quantum Hardware Noise

Deterministic
Barren plateau

Vettori di Località

Dato uno spazio di Hilbert di più qbit ($H = \otimes_m H_m$, con $\dim(\mathcal{H}) = \prod_m \dim(\mathcal{H}_m)$), Chiamiamo $\Phi_{\vec{\theta}}$ il canale quantistico (circuit) che agisce sui qbit e dipende da un vettore di parametri $\vec{\theta}$.

La funzione di costo che si vuole in genere minimizzare è: $\mathcal{L}_{\rho, k} = \text{Tr}[\Phi(\rho)H]$, con H osservabile da determinare. Il BP può essere visto dalla varianza di $\mathcal{L} : \mathbf{V}_{\rho, H}(\theta)$. Nello specifico, c'è un BP se $\mathbf{V} \propto O(e^{-\beta n})$ con n numero di QBIT.

In generale, lo spazio H può essere suddiviso in sottosistemi locali. Questa suddivisione di H induce una corrispondente partizione dello spazio B degli operatori limitati su H . Gli operatori in B vengono raggruppati in sottospazi B_{κ} . Ogni sottospazio è identificato da un vettore binario κ (con $\kappa \in \{0, 1\}^M$), che indica su quali sottosistemi locali gli operatori agiscono trivialmente e su quali non (per ogni sottospazio non triviale $\kappa_m = 1$).

Lo spazio B è scomposto come somma diretta:

$$B = \bigoplus_{\kappa \in \{0,1\}^M} B_{\kappa}.$$

La dimensione di ciascun sottospazio B_{κ} è data da: $d_{\kappa} = \prod_m (d_m^2 - 1)^{\kappa_m}$, dove d_m rappresenta la dimensione del sottosistema H_m . Il caso speciale $\kappa = 0$ corrisponde a operatori triviali.

Possiamo in sostanza dividere lo spazio di Hilbert in base a come gli operatori agiscono su di esso. Se, per esempio, un operatore A agisce trivialmente sul primo Qbit e non trivialmente sul secondo, lo indicheremo con la stringa: "10". Come vedremo più avanti, gli operatori faranno nascere dei sottospazi isolati nello spazio di Hilbert, ovvero, sottospazi dove gli stati rimangono anche dopo l'applicazione lineare. Allo stesso tempo, è possibile osservare sottospazi che

si disperdono, ovvero sottospazi i cui elementi migrando in altri sottospazi dopo l'applicazione lineare. Questa osservazione relativa alla dispersione e all'assorbimento di stati è alla base di questo lavoro, in quanto il trasferimento di informazione può prevenire la formazione di BP.

Un operatore $A \in B$ si dice κ -locale se appartiene a B_κ . A ciascun operatore si associa un vettore di località $\ell_A \in \mathbb{R}^{2^M}$, che descrive quanto l'operatore è distribuito nei vari sottospazi B_κ . Il vettore di località ℓ_A è definito come un vettore di dimensione pari al numero di sottospazi in cui lo spazio B è suddiviso. Ogni componente $(\ell_A)_\kappa$ rappresenta la proiezione dell'operatore A sul sottospazio B_κ .

Questa formula misura la somma dei quadrati delle proiezioni dell'operatore A sugli elementi della base $\{B_j\}_j$. Questo vettore è calcolato utilizzando una base ortonormale hermitiana $\{B_j\}_j$ di B_κ , e gli elementi di ℓ_A sono definiti come:

$$(\ell_A)_\kappa = \sum_{j=1}^{d_\kappa} \text{Tr}[B_j A]^2.$$

Se A è un operatore totalmente locale, il vettore di località assume la forma più semplice: $(\ell_A)_\lambda = \delta_{\kappa,\lambda} \|A\|_2$.

Locality transfer Matrix (LTM)

A questo punto siamo interessati a capire quando e come un operatore lineare che agisce su B preserva la località dello spazio, cioè quanto due sottospazi di B sono messi in comunicazione da un operatore lineare. Questo è rappresentato dalla matrice di trasferimento di località T , i cui elementi sono definiti come:

$$T_{\kappa,\lambda} = \frac{1}{d_\kappa} \sum_{j=1}^{d_\kappa} (\ell_{\Lambda(B_j)})_\lambda,$$

dove $\ell_{\Lambda(B_j)}$ è il vettore di località associato all'immagine di B_j sotto Λ .

La matrice T può essere organizzata in blocchi per evidenziare meglio le interazioni tra gruppi di sottospazi. Ad esempio, se i sottospazi B_κ sono raggruppati in modo gerarchico, la matrice T può essere scritta come:

$$T = \begin{bmatrix} T^{(1,1)} & 0 & \dots & T^{(1,n)} \\ 0 & T^{(2,2)} & \dots & T^{(2,n)} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & T^{(n,n)} \end{bmatrix},$$

dove i blocchi diagonali $T^{(i,i)}$ rappresentano la preservazione della località all'interno di ciascun gruppo di sottospazi, mentre l'ultima colonna rappresenta la nascita del fenomeno di assorbimento.

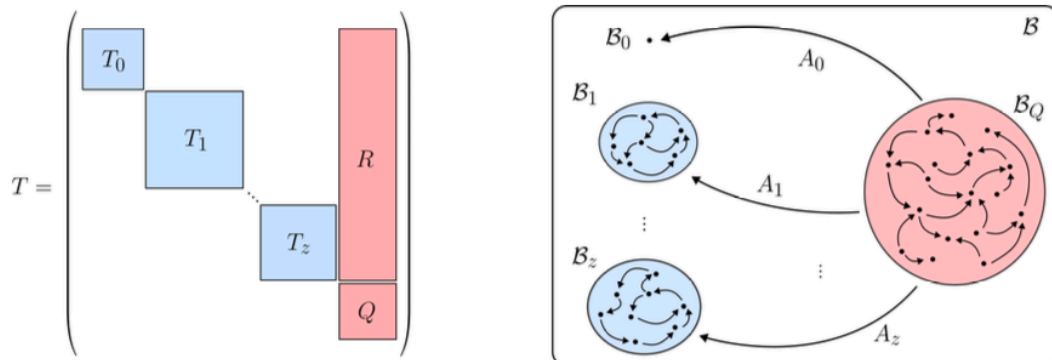
L'idea più immediata della matrice T è quella di una matrice che cattura quanto ciascuna località viene spostata in un'altra. Per certi versi, possiamo attribuire alle colonne della matrice un significato di proiettore.

Da un punto di vista pratico, per tutti i gate unitari quantistici, la matrice LTM è definita positiva e scomponibile in blocchi con una parte irriducibile.

Ovviamente il sottospazio totalmente locale, forma sempre un blocco isolato, l'unico della sua riga e colonna.

Ogni blocco diagonale della LTM è un sottospazio di Hilbert dove vivono i qbit, nel quale l'operatore lineare conserva gli elementi. Gli elementi della colonna irriducibile A sono le parti del sottospazio di Hilbert che sono trasferite ad altri blocchi. Questo fenomeno prende il nome di assorbimento. In figura, è rappresentata la generica LTM.

```
In [6]: from IPython.display import Image, display
image3 = "Img/Screenshot 2025-01-22 alle 11.21.30.png"
display(Image(filename=image3, width=1000, height=600))
```



Varianza Analitica

L è il numero dei layers. $E_{\theta}\{. \}$ rappresenta il valore di aspettazione rispetto al parametro θ .

Vale:

$$E_{\theta}(\text{Tr}[\Phi_{\theta}(\rho)H]^2) = (l_{\rho}, \prod_{l=1}^L T_l l_H)$$

, dove T_l è la LTM associata al canale $\mathcal{E}_l$ (matrice non-negativa). Inoltre:

$$\mathbf{V}_{\rho, H}^L = (l_{\rho}, \prod_{l=1}^L T_l l_H) - \frac{\text{Tr}[H]^2}{d^2}.$$

Questa formula è inutilizzabile per grandi profondità a causa della sua complessità computazionale; ciò nondimeno sarà usata per ricavare il limite a profondità grandi.

Considereremo circuiti dove $\mathcal{E}_l = \mathcal{E}$ e dove tutti gli osservabili H hanno traccia nulla, in modo che il secondo termine della varianza sia uguale a zero e che scompaia la sommatoria in favore di L .

$$\mathbf{V}_{\rho, H}^L = (l_\rho, \prod_{l=1}^L T_l l_H)$$

Dove T_l è la matrice TLM del gate $\mathcal{E}_l$

Se $L \rightarrow \infty$,

$$V_{\infty}^{\rho, H} = \sum_z (\ell_{\rho_z, w_z})(\ell_H)_z + (\ell_{\rho_z, w_z})(A \ell_H)_z$$

Dove w_z è autovalore destro di T_z , A è la **matrice di assorbimento**, $A = R(1 - Q)^{-1}$ (Vedi Matrice di trasferimento di località).

Questa forma mette in evidenza come certi sottospazi si comportino da attrattori per la varianza.

In generale, non è detto che la varianza converga ad un valore ma, almeno rimane limitata. Nel lavoro, la convergenza della varianza sarà studiata dalla limitazione:

$$|V_{\rho, H}^L - V_{\rho, H}^{\infty}| \in O\left(e^{-\beta L} \|H\|_2^2\right)$$

Noise non unitario

Adesso prendiamo in considerazione circuiti quantistici affetti da noise non unitale, come per esempio un rumore depolarizzante. Consideriamo $\mathcal{E}_c(\rho) = (1 - p)\mathcal{E}(\rho) + p\tilde{\rho}$ dove $\tilde{\rho}$ è uno stato arbitrario entangled e p la probabilità di errore.

Possiamo pensare a $\Phi(\rho) = (1 - p)\rho + p\tilde{\rho}$. Allora:

$$V_{\infty}^{\rho, H} = p^2(l_{\tilde{\rho}}(I - (1 - p)^2 T)l_H)$$

e se T è un proiettore,

$$V_{\infty}^{\rho, H} = (p/(2 - p) - p^2)(l_{\tilde{\rho}}, T l_H) + p^2(l_{\tilde{\rho}}, l_H)$$

Notiamo che è scomparsa la dipendenza dallo stato iniziale e solo i layer dopo un certo L contribuiscono sensibilmente alla varianza.

In particolare, possiamo considerare due limiti contrapposti. Nel primo il noise "entangla" lentamente lo stato ($T \simeq I$), la dipendenza da $p \ll 1$ svanisce. D'altra parte, se il noise entangla velocemente lo stato, il secondo termine è dominante.

Il grafico sopra è tratto dal paper da cui siamo partiti. Mostra la differenza tra gli andamenti della varianza calcolati analiticamente e numericamente per un circuito con "slow entangling" e "rapid entangling". Il nostro obiettivo sarà di riprodurre risultati simili.

Circuiti scelti

Per simulare il rumore entangling, il sistema è stato mandato, ad ogni layer, con probabilità p lo stato nel GHZ (stato di massimo entanglement):

$$\tilde{\rho} = \frac{(|0\rangle^{\otimes n} + |1\rangle^{\otimes n})(\langle 0|^{\otimes n} + \langle 1|^{\otimes n})}{2^n}$$

.

Il canale considerato è quindi della forma $\mathcal{E}_c = N \circ \mathcal{E}$, dove:

$$\mathcal{E}_c(\rho) = (1 - p)\mathcal{E}(\rho) + p\tilde{\rho}$$

,

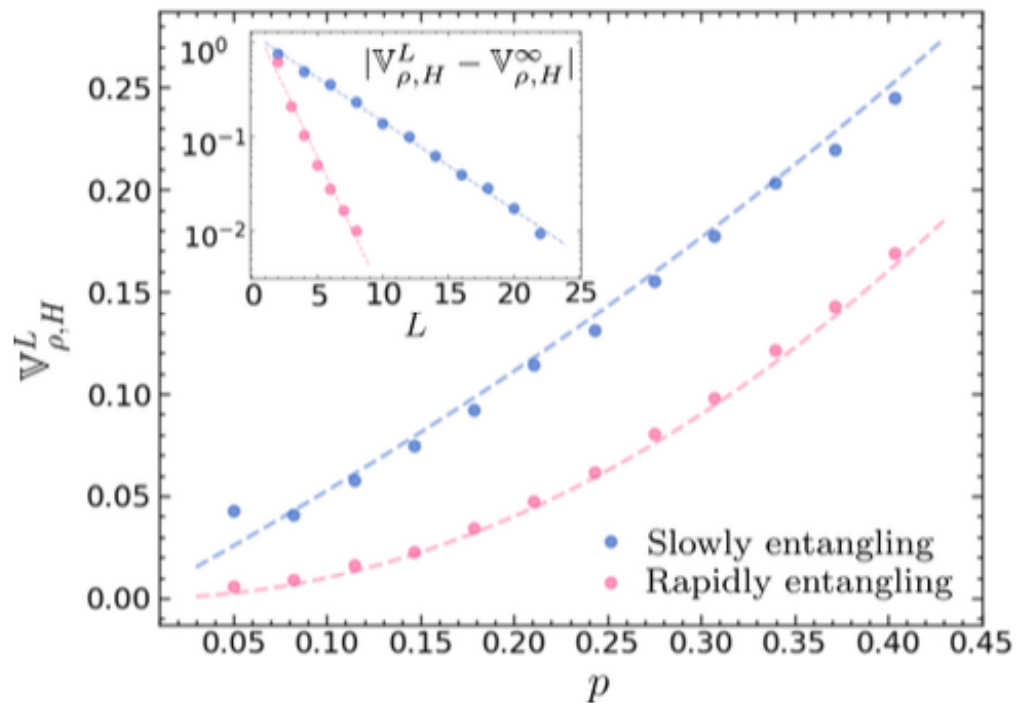
Lo stato iniziale è fissato come $\rho = (|0\rangle\langle 0|)^{\otimes n}$ a cui vengono applicate una serie di rotazioni su X e Z casuali per riprodurre un 2-design.

L'osservabile scelto è stato:

$$H = h \sum_{k=1}^n 2^{n/2} Z_k \otimes Z_{k+1},$$

In [7]: `from IPython.display import Image, display`

```
# Percorso dell'immagine
image4 = 'Img/Screenshot 2025-01-22 alle 16.47.42.png'
display(Image(filename=image4, width=500, height=400))
```



Fast Entangling

Per un circuito fast entangling, è applicata allo stato iniziali una catena di CNOT, come nella figura sotto. Successivamente si introduce la mappa di rumore. Il tutto viene ripetuto per L layers al termine dei quali si misura lo stato e l'osservabile.

L'osservabile scelto è stato:

```
In [8]: import qiskit as qc
from qiskit_aer import Aer
import numpy as np
from qiskit import QuantumCircuit
from qiskit.quantum_info import DensityMatrix, SparsePauliOp
import matplotlib.pyplot as plt
import itertools
import matplotlib.pyplot as plt
from collections import defaultdict
from scipy.sparse.csgraph import connected_components
from functools import reduce
from qiskit.circuit import Gate

def generate_random_parameters(n):
    return 2 * np.pi * np.random.rand(2 * n)
n = 4 # Numero di qubit
L_final=2
noise_gate = Gate(name="Noise", num_qubits=n, params=[])
circuit = QuantumCircuit(n)
rho = DensityMatrix.from_instruction(circuit).data # Matrice densità

for _ in range(L_final):
    # Applica rotazioni casuali
    parameters = generate_random_parameters(n)
```

```

for i in range(n):
    circuit.rx(parameters[2 * i], i)
    circuit.rz(parameters[2 * i + 1], i)
circuit.cx(0,1)
for i in range (n-2):
    circuit.cx(i+1, i+2)
    circuit.cx(i, i+1)
circuit.cx(n-2,n-1)

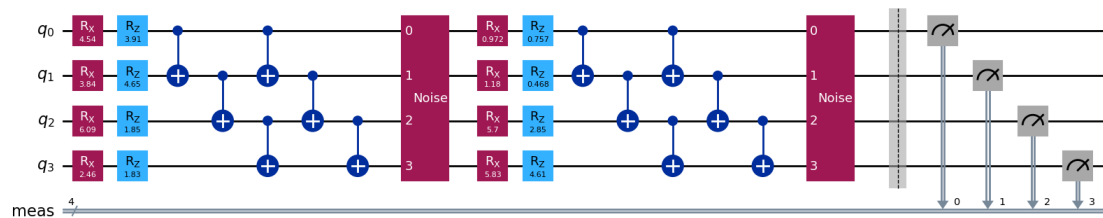
circuit.append(noise_gate, range(n))

circuit.measure_all()

circuit.draw('mpl')

```

Out[8]:



Slow Entangling

Per quanto riguarda lo "slow entangle", al posto della sequenza di Cnot e SWAP, è stata applicata una catena di rotazioni controllate "piccole" lungo X.

```

In [9]: from qiskit.circuit import Gate
import numpy as np
from qiskit import QuantumCircuit
from qiskit.quantum_info import DensityMatrix

L_final = 2
noise_gate = Gate(name="Noise", num_qubits=n, params=[])

circuit = QuantumCircuit(n)
rho = DensityMatrix.from_instruction(circuit).data # Matrice densi

for _ in range(L_final):
    # Applica rotazioni casuali
    parameters = 2 * np.pi * np.random.rand(2 * n)
    for i in range(n):
        circuit.rx(parameters[2 * i], i)
        circuit.rz(parameters[2 * i + 1], i)

    for i in range (n-1):
        circuit.crx(np.pi / 20, i, i+1)

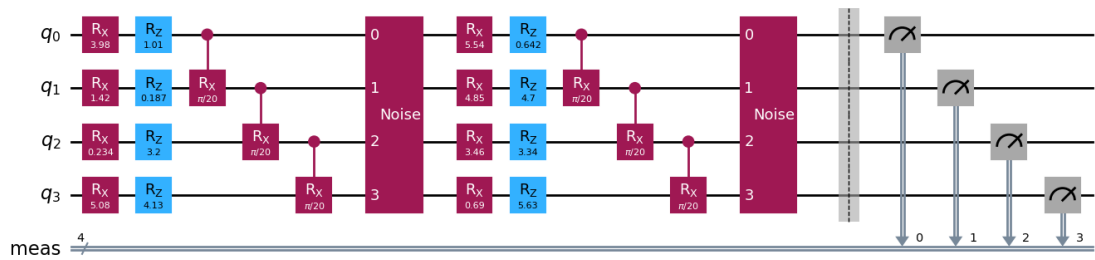
    # Aggiunta del rumore
    circuit.append(noise_gate, range(n))

circuit.measure_all()

```

```
# Disegna il circuito
circuit.draw('mpl')
```

Out[9]:



```
In [10]: from qiskit.quantum_info import Operator
from inspect import trace
import numpy as np
import itertools
import matplotlib.pyplot as plt
from collections import defaultdict
from scipy.sparse.csgraph import connected_components
import qiskit as qc
from qiskit_aer import Aer
import numpy as np
from qiskit import QuantumCircuit
from qiskit.quantum_info import DensityMatrix, SparsePauliOp

n = 4 # Numero di qubit

def pauli_matrices_divided():
    sqrt2 = np.sqrt(2)
    return {
        'I': np.array([[1, 0], [0, 1]], dtype=complex) / sqrt2,
        'X': np.array([[0, 1], [1, 0]], dtype=complex) / sqrt2,
        'Y': np.array([[0, -1j], [1j, 0]], dtype=complex) / sqrt2,
        'Z': np.array([[1, 0], [0, -1]], dtype=complex) / sqrt2
    }

# Creazione delle matrici di Pauli divise
pauli_matrices = pauli_matrices_divided()

# Definizione delle etichette delle matrici di Pauli
matrix_labels = list(pauli_matrices.keys()) # ['I', 'X', 'Y', 'Z']

# 2. Genera tutti i possibili prodotti tensore e definisce la "binary tuple"
tensor_products = []
for combination in itertools.product(matrix_labels, repeat=n):
    tensor = pauli_matrices[combination[0]]
    for label in combination[1:]:
        tensor = np.kron(tensor, pauli_matrices[label])
    binary_tuple = tuple(1 if label != 'I' else 0 for label in combination)
    tensor_products.append((binary_tuple, tensor))

# Ordinamento in base al valore della tuple (facoltativo)
tensor_products.sort(key=lambda item: sum(bit * (2**i) for i, bit in enumerate(item)))

# Raggruppa i tensori in base alla binary tuple
groups = defaultdict(list)
```

```

for binary_tuple, tensor in tensor_products:
    groups[binary_tuple].append(tensor)

# Lista ordinata delle tuple uniche e definizione del numero J di g
unique_tuples = sorted(groups.keys(), key=lambda tup: sum(bit * (2**
J = len(unique_tuples)

#####_-----
# 3. Definizione della matrice H (in questo caso 4 gate pauli X)

# Applica rotazioni casuali
parameters = 2 * np.pi * np.random.rand(2 * n)
circuit = QuantumCircuit(n)

circuit.cx(0,1)
for i in range (n-2):
    circuit.cx(i+1, i+2)
    circuit.cx(i, i+1)
circuit.cx(2,3)

H=Operator(circuit)
H=np.array(H.data)
#-----

# 4 Costruzione della matrice T e di l:
# Per ciascun gruppo j (per tensori B) e per ciascun gruppo k (per
# si somma (con normalizzazione) il contributo: (trace(H† B H P))^2

T = np.zeros((J, J), dtype=complex)

for j_idx, tup_j in enumerate(unique_tuples):
    LT_vec = np.zeros(J, dtype=complex)
    # Calcola il fattore di normalizzazione per il gruppo B: per og
    norm_factor_B = 3 ** (sum(tup_j))

    # Per ogni tensore B nel gruppo j
    for B in groups[tup_j]:
        # Ciclo sui gruppi per P (indice k)
        for k_idx, tup_k in enumerate(unique_tuples):
            trace_sum = 0
            # Fattore di normalizzazione per il gruppo P
            norm_factor_P = 3 ** (sum(tup_k))
            for P in groups[tup_k]:
                trace_val = np.trace(H.conj().T @ B @ H @ P)
                # Normalizzazione: il contributo viene diviso per i
                trace_sum += (trace_val**2) / (norm_factor_B )
            LT_vec[k_idx] += trace_sum
    T[j_idx, :] = LT_vec

```

```
# 5. Diagonalizzazione a blocchi fortemente connessi tramite permutazione
# Costruiamo una "matrice di connettività": consideriamo un arco tra due nodi se la
# matrice di adiacenza ha un valore non nullo
tol = 1e-16
connectivity = (np.abs(T) > tol).astype(int)

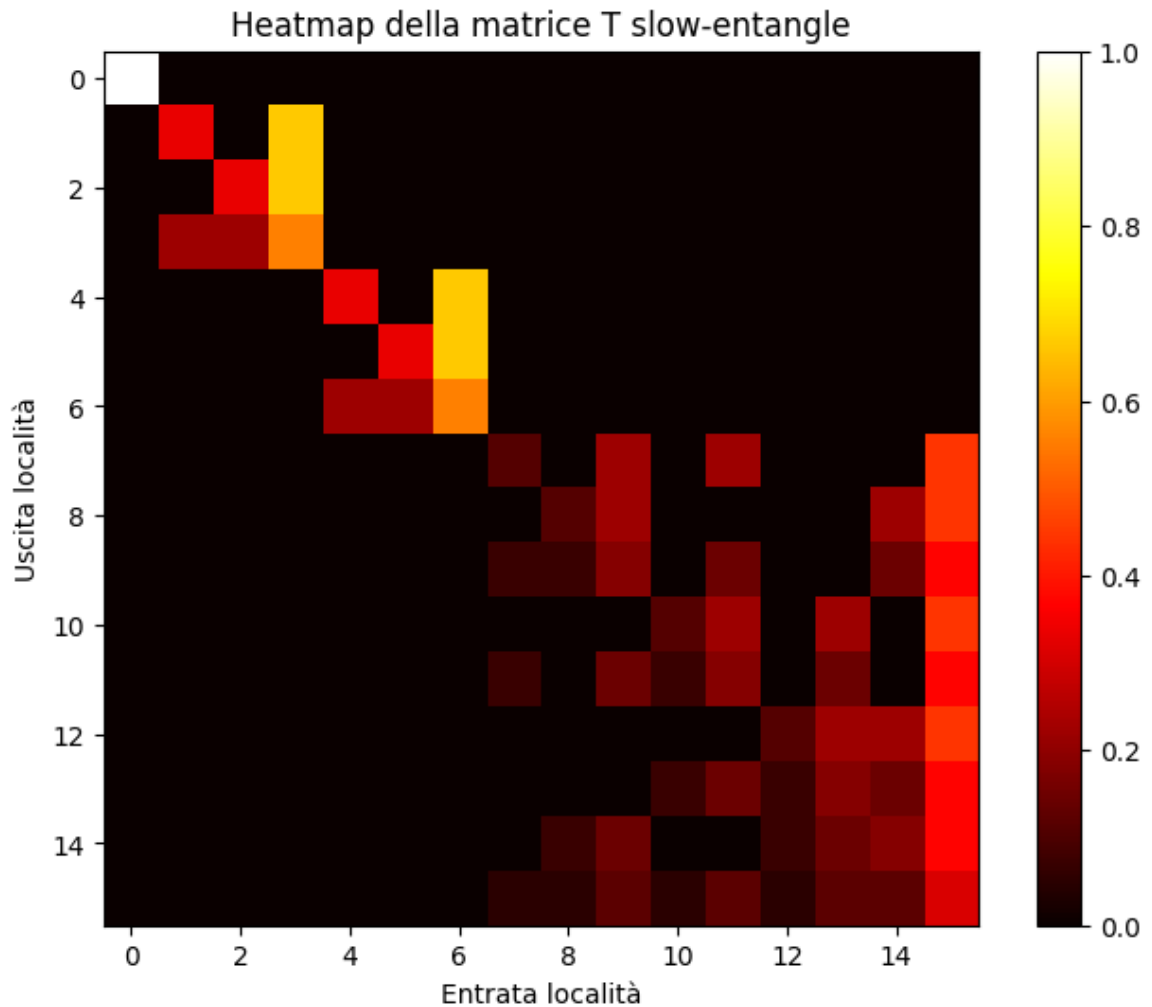
# Trova i componenti connessi (trattando la matrice come grafo non orientato)
n_components, comp_labels = connected_components(connectivity, directed=False)

# La permutazione che raggruppa per componente è data da:
permutation = np.argsort(comp_labels)
# Riordiniamo T per ottenere una forma bloc-diagonale
T_perm = T[permutation, :][:, permutation]

# Salviamo la mappatura della permutazione: (nuovo indice -> indice originale)
perm_mapping = {new_idx: old_idx for new_idx, old_idx in enumerate(permutation)}

# 6. Plot della matrice T permutata come heatmap (si plottano i valori reali)
plt.figure(figsize=(8, 6))
plt.imshow(np.real(T_perm), cmap='hot', interpolation='nearest')
plt.colorbar()
plt.xlabel("Entrata località")
plt.ylabel("Uscita località")
plt.title("Heatmap della matrice T slow-entangle")
plt.show()

# Salva (o stampa) la mappatura della permutazione
print("Mappatura della permutazione (nuovo indice -> indice originale)")
print(perm_mapping)
```



Mappatura della permutazione (nuovo indice $\rightarrow$ indice originale):
 $\{0: 0, 1: 1, 2: 4, 3: 5, 4: 2, 5: 8, 6: 10, 7: 3, 8: 6, 9: 7, 10: 9, 11: 11, 12: 12, 13: 13, 14: 14, 15: 15\}$

Possiamo notare che, nonostante la dimensione ridotta del sistema, la cascata di Cnot tende a creare subito un entangle molto forte. Nonostante ciò, per soli 4 Qbit ci sono ancora sottospazi isolati.

```
In [11]: from inspect import trace
import numpy as np
import itertools
import matplotlib.pyplot as plt
from collections import defaultdict
from scipy.sparse.csgraph import connected_components
import qiskit as qc
from qiskit_aer import Aer
import numpy as np
from qiskit import QuantumCircuit
from qiskit.quantum_info import DensityMatrix, SparsePauliOp

n = 4 # Numero di qubit

def pauli_matrices_divided():
    sqrt2 = np.sqrt(2)
    return {
        'I': np.array([[1, 0], [0, 1]], dtype=complex) / sqrt2,
        'X': np.array([[0, 1], [1, 0]], dtype=complex) / sqrt2,
```

```

        'Y': np.array([[0, -1j], [1j, 0]], dtype=complex) / sqrt2,
        'Z': np.array([[1, 0], [0, -1]], dtype=complex) / sqrt2
    }

# Creazione delle matrici di Pauli divise
pauli_matrices = pauli_matrices_divided()

# Definizione delle etichette delle matrici di Pauli
matrix_labels = list(pauli_matrices.keys()) # ['I', 'X', 'Y', 'Z']

# 2. Genera tutti i possibili prodotti tensore e definisce la "binary tuple"
tensor_products = []
for combination in itertools.product(matrix_labels, repeat=n):
    tensor = pauli_matrices[combination[0]]
    for label in combination[1:]:
        tensor = np.kron(tensor, pauli_matrices[label])
    binary_tuple = tuple(1 if label != 'I' else 0 for label in combination)
    tensor_products.append((binary_tuple, tensor))

# Ordinamento in base al valore della tuple (facoltativo)
tensor_products.sort(key=lambda item: sum(bit * (2**i) for i, bit in enumerate(item)))

# Raggruppa i tensori in base alla binary tuple
groups = defaultdict(list)
for binary_tuple, tensor in tensor_products:
    groups[binary_tuple].append(tensor)

# Lista ordinata delle tuple uniche e definizione del numero J di gruppi
unique_tuples = sorted(groups.keys(), key=lambda tup: sum(bit * (2**i) for i, bit in enumerate(tup)))
J = len(unique_tuples)

#####_-----
# 3. Definizione della matrice H (in questo caso 4 gate pauli X)

def pauli_x():
    return np.array([[0, 1], [1, 0]], dtype=complex)

# Prodotto tensoriale di 4 porte X
x_gate = pauli_x()
H = x_gate
for _ in range(3): # Aggiunge 3 porte X per un totale di 4
    H = np.kron(H, x_gate)

#-----

# 4 Costruzione della matrice T e di l:
# Per ciascun gruppo j (per tensori B) e per ciascun gruppo k (per P)
# si somma (con normalizzazione) il contributo: (trace(H† B H P))^2

T = np.zeros((J, J), dtype=complex)

```

```

for j_idx, tup_j in enumerate(unique_tuples):
    LT_vec = np.zeros(J, dtype=complex)
    # Calcola il fattore di normalizzazione per il gruppo B: per ogni gruppo B
    norm_factor_B = 3 ** (sum(tup_j))

    # Per ogni tensore B nel gruppo j
    for B in groups[tup_j]:
        # Ciclo sui gruppi per P (indice k)
        for k_idx, tup_k in enumerate(unique_tuples):
            trace_sum = 0
            # Fattore di normalizzazione per il gruppo P
            norm_factor_P = 3 ** (sum(tup_k))
            for P in groups[tup_k]:
                trace_val = np.trace(H.conj().T @ B @ H @ P)
                # Normalizzazione: il contributo viene diviso per il prodotto dei fattori
                trace_sum += (trace_val**2) / (norm_factor_B * norm_factor_P)
            LT_vec[k_idx] += trace_sum
    T[j_idx, :] = LT_vec

# 5. Diagonalizzazione a blocchi fortemente connessi tramite permutazione
# Costruiamo una "matrice di connettività": consideriamo un arco tra i nodi se la traccia è maggiore di una soglia
tol = 1e-16
connectivity = (np.abs(T) > tol).astype(int)

# Trova i componenti connessi (trattando la matrice come grafo non orientato)
n_components, comp_labels = connected_components(connectivity, directed=False)

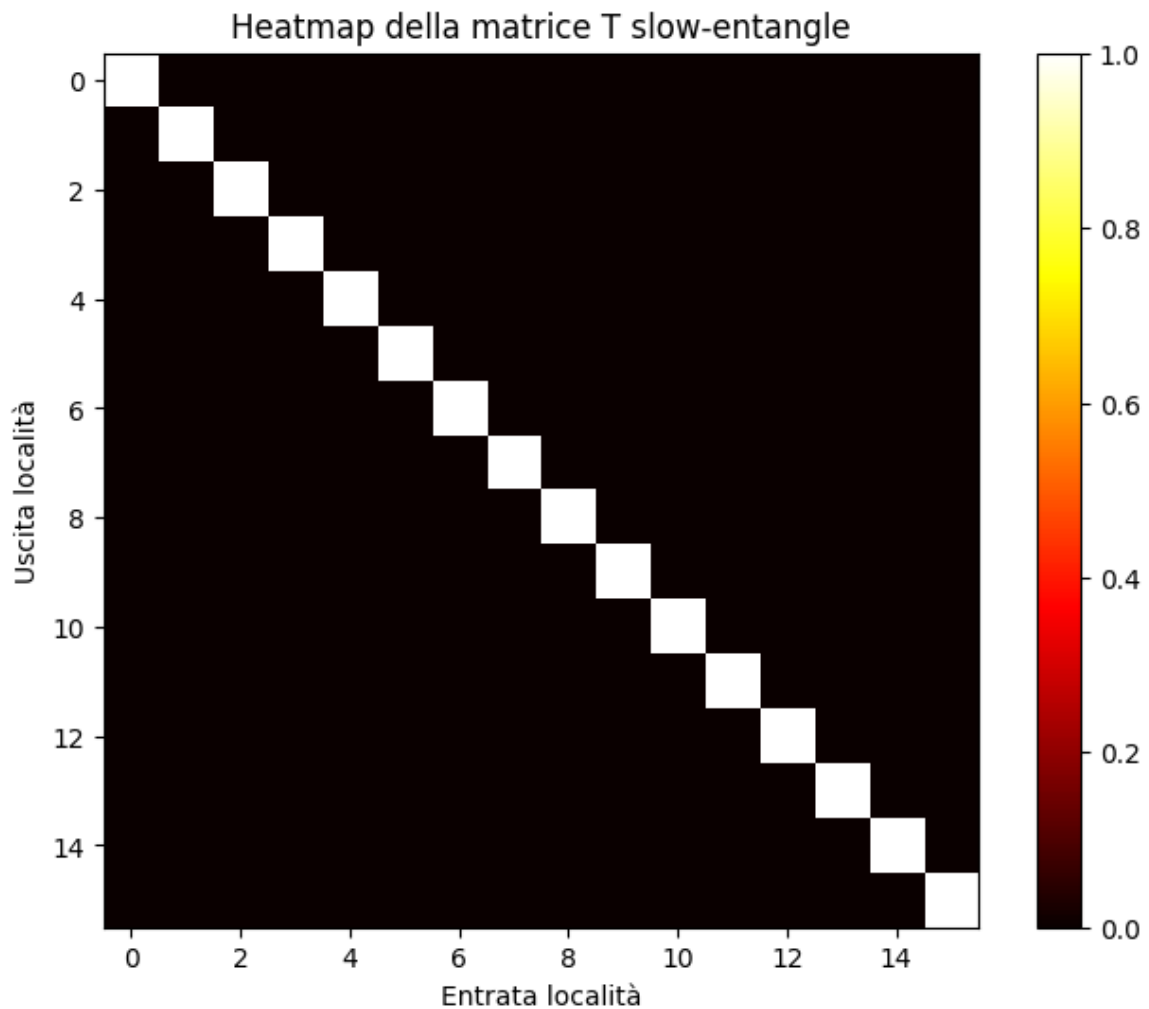
# La permutazione che raggruppa per componente è data da:
permutation = np.argsort(comp_labels)
# Riordiniamo T per ottenere una forma bloc-diagonale
T_perm = T[permutation, :][:, permutation]

# Salviamo la mappatura della permutazione: (nuovo indice -> indice originale)
perm_mapping = {new_idx: old_idx for new_idx, old_idx in enumerate(permutation)}

# 6. Plot della matrice T permutata come heatmap (si plottano i valori reali)
plt.figure(figsize=(8, 6))
plt.imshow(np.real(T_perm), cmap='hot', interpolation='nearest')
plt.colorbar()
plt.xlabel("Entrata località")
plt.ylabel("Uscita località")
plt.title("Heatmap della matrice T slow-entangle")
plt.show()

# Salva (o stampa) la mappatura della permutazione
print("Mappatura della permutazione (nuovo indice -> indice originale):")
print(perm_mapping)

```



Mappatura della permutazione (nuovo indice $\rightarrow$ indice originale):
 $\{0: 0, 1: 1, 2: 2, 3: 3, 4: 4, 5: 5, 6: 6, 7: 7, 8: 8, 9: 9, 10: 10, 11: 11, 12: 12, 13: 13, 14: 14, 15: 15\}$

Al contrario, un entangle debole mantiene un isolamento forte tra i sottospazi.
 Quindi la LTM ha una forma simile all'applicazione non triviale.

Le formule utili per calcolare le varianze, contengono le LTM dei gate unitari del circuito quantistico.

```
In [12]: from inspect import trace
import numpy as np
import itertools
import matplotlib.pyplot as plt
from collections import defaultdict
from scipy.sparse.csgraph import connected_components
import qiskit as qc
from qiskit_aer import Aer
import numpy as np
from qiskit import QuantumCircuit
from qiskit.quantum_info import DensityMatrix, SparsePauliOp, Operator

# Parametri
n=4
shots = 1000 # Numero di esecuzioni
```

```

L_values_range = 25 # Numero massimo di blocchi di operazioni
p = 0.1 # Probabilità di applicare lo stato GHZ
h = 1 # Coefficiente nell'Hamiltoniano

# Funzione per generare parametri casuali
def generate_random_parameters(n):
    return 2 * np.pi * np.random.rand(2 * n)

# Funzione per mescolare rho con lo stato GHZ

def rho_noise(p, rho):
    rho_ghz = np.zeros_like(rho)
    rho_ghz[0, 0] = 0.5
    rho_ghz[-1, -1] = 0.5
    rho_ghz[0, -1] = 0.5
    rho_ghz[-1, 0] = 0.5
    return (1-p)* rho + (p) * rho_ghz

# Funzione per costruire il circuito e applicare il rumore
def circuit_with_noise(n, p, L):
    circuit = QuantumCircuit(n)
    rho = DensityMatrix.from_instruction(circuit).data # Matrice d
    # Applica rotazioni casuali
    parameters = generate_random_parameters(n)
    for i in range(n):
        circuit.rx(parameters[2 * i], i)
        circuit.rz(parameters[2 * i + 1], i)
    circuit.cx(0,1)
    for i in range (n-2):
        circuit.cx(i+1, i+2)
        circuit.cx(i, i+1)
    circuit.cx(n-2,n-1)

    W=np.array(qc.quantum_info.Operator(circuit))
    for _ in range(L):
        # Evoluzione dello stato
        rho = W @ rho @ W.conj().T
        rho = rho_noise(p, rho)
    return rho

def circuit_with_noise_slow(n, p, L):
    circuit = QuantumCircuit(n)
    rho1 = DensityMatrix.from_instruction(circuit).data

    for i in range(n-1):
        parameters = generate_random_parameters(n)
        circuit.rx(parameters[2 * i], i)
        circuit.rz(parameters[2 * i + 1], i)
    # Applica i CNOT e gli swap
    circuit.crx(np.pi / 20, 0, 1)
    circuit.crx(np.pi / 20, 1, 2)
    circuit.crx(np.pi / 20, 2, 3)

    W=np.array(qc.quantum_info.Operator(circuit))
    for _ in range(L):# Evoluzione dello stato
        rho1 = W @ rho1 @ W.conj().T

```

```

        rho1 = rho_noise(p, rho1)
    return rho1

# Creazione dell'Hamiltoniano W
pauli_strings = []
coefficients = []

for k in range(n-1):
    # Coefficiente del termine
    coefficients.append(h * (2 * (n/2)))
    # Stringa di Pauli per Z_k ⊗ Z_{k+1}
    pauli_string = ['I'] * n
    pauli_string[k] = 'Z' # Z_k
    pauli_string[(k + 1)] = 'Z' # Z_{k+1} (periodico)
    pauli_strings.append(''.join(pauli_string))

# Creazione dell'Hamiltoniano come SparsePauliOp
W = SparsePauliOp(pauli_strings, coeffs=np.array(coefficients))

# Lista per memorizzare i valori di L
L_values_all = {L: [] for L in range(1, L_values_range + 1)}
L_values_all1 = {L: [] for L in range(1, L_values_range + 1)}

for L in range(1, L_values_range + 1):
    L_values_shots = []
    L_values_shots1 = []

    for _ in range(shots):
        rho = circuit_with_noise(n, p, L) # Stato con rumore
        W_matrix = W.to_matrix()
        L_value = np.trace(W_matrix @ rho.data) # Loss
        L_values_shots.append(L_value)

        rho1 = circuit_with_noise_slow(n, p, L) # Stato con rumore
        W_matrix = W.to_matrix()
        L_value1 = np.trace(W_matrix @ rho1.data) # Loss
        L_values_shots1.append(L_value1)

    L_values_all[L] = L_values_shots
    L_values_all1[L] = L_values_shots1

L_means = {L: np.mean(L_values_all[L]) for L in range(1, L_values_range + 1)}
L_variances = {L: np.var(L_values_all[L]) for L in range(1, L_values_range + 1)}
L_means1 = {L: np.mean(L_values_all1[L]) for L in range(1, L_values_range + 1)}
L_variances1 = {L: np.var(L_values_all1[L]) for L in range(1, L_values_range + 1)}

# Plot della varianza in funzione di L
L_range = list(L_variances.keys())
var_values = list(L_variances.values())
L_values = list(L_means.values())
var_values1 = list(L_variances1.values())
L_values1 = list(L_means1.values())
var_values1 = np.array(var_values1, dtype=float)

plt.figure(figsize=(10, 5))

```

```

plt.plot(L_range, var_values, marker='o', linestyle='--', color='b')
plt.plot(L_range, var_values1, marker='o', linestyle='--', color='g')
plt.axhline(y=2*53.3*p**2, color='b', linestyle='--', label="Linea")
plt.axhline(y= (p/(2-p))*53.3, color='g', linestyle='--', label="L")
plt.xlabel('L (Numero di blocchi di operazioni)')
plt.ylabel('Varianza della Loss')
plt.title('Varianza della Loss in funzione di L')
plt.grid(True)
plt.show()

var_values_array = np.array(list(L_variances.values()), dtype=float)
var_values_array_1 = np.array(list(L_variances1.values()), dtype=float)
# Applica le correzioni
var_values_log = var_values_array - (10 * 5.33 * p**2)
var_values_log_1 = var_values_array_1 - ((p / (2 - p)) * 53.3)

p_values = np.linspace(0.05, 0.5, 20)

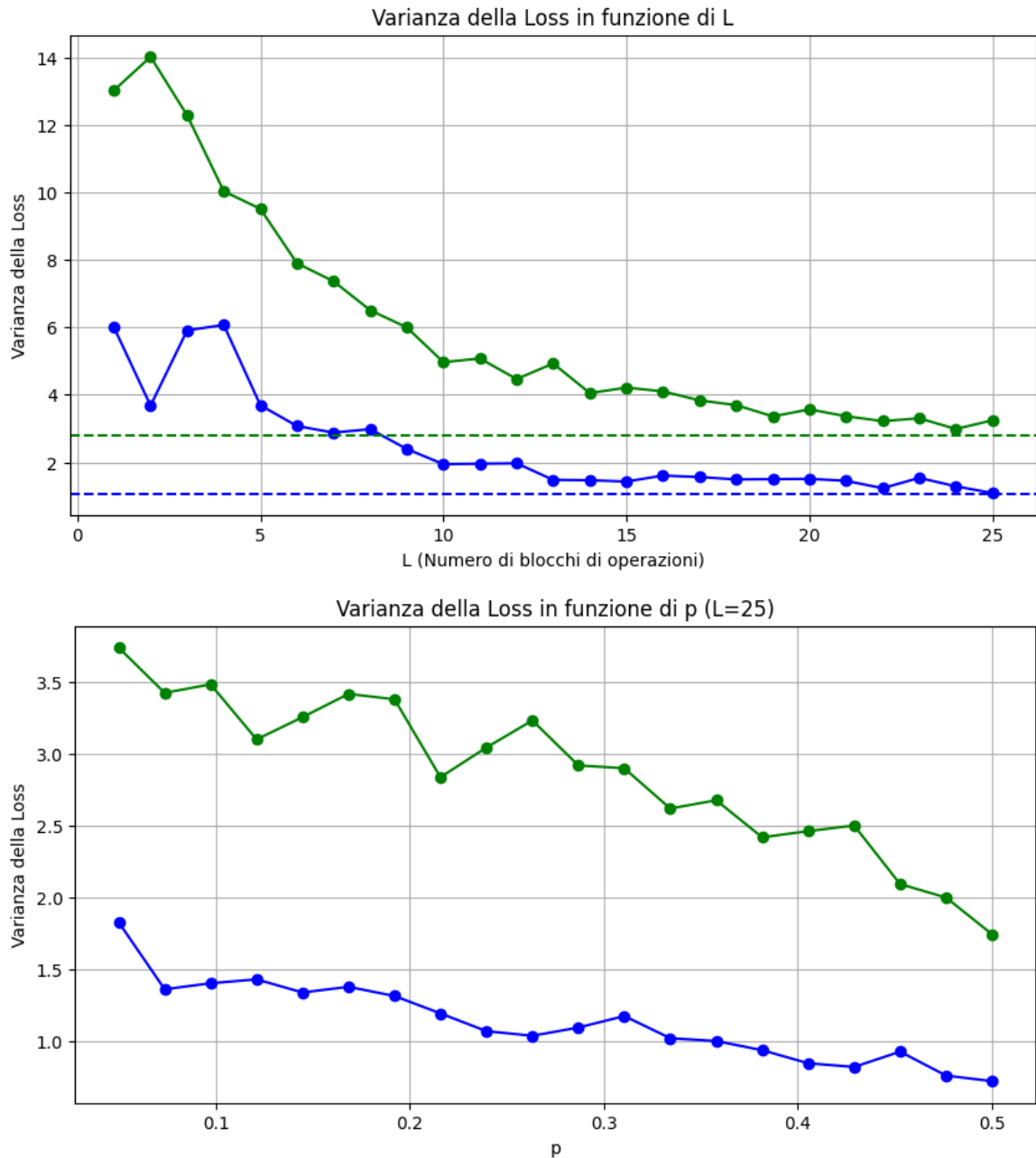
for p in p_values:
    L_values_shots = []
    L_values_shots1 = []
    for _ in range(shots):
        rho = circuit_with_noise(n, p, L_values_range)
        rho1=circuit_with_noise_slow(n,p, L_values_range) # Stato
        W_matrix = W.to_matrix()
        L_value = np.trace(W_matrix @ rho.data)
        L_value1 = np.trace(W_matrix @ rho1.data) # Loss
        L_values_shots.append(L_value)
        L_values_shots1.append(L_value1)
    L_values_all[p] = L_values_shots
    L_values_all1[p] = L_values_shots1

L_means = {p: np.mean(L_values_all[p]) for p in p_values}
L_variances = {p: np.var(L_values_all[p]) for p in p_values}
L_means1 = {p: np.mean(L_values_all1[p]) for p in p_values}
L_variances1 = {p: np.var(L_values_all1[p]) for p in p_values}

var_values = list(L_variances.values())
L_values = list(L_means.values())
var_values1 = list(L_variances1.values())
L_values1 = list(L_means1.values())
var_values1 = np.array(var_values1, dtype=float)

# Plot della varianza in funzione di p
plt.figure(figsize=(10, 5))
plt.plot(p_values, var_values, marker='o', linestyle='--', color='b')
plt.plot(p_values, var_values1, marker='o', linestyle='--', color='g')
plt.xlabel('p')
plt.ylabel('Varianza della Loss')
plt.title(f'Varianza della Loss in funzione di p (L={L_values_range}')
plt.grid(True)
plt.show()

```



Notiamo che l'andamento in funzione di L è esponenziale, coerente con quello previsto. Le linee tratteggiate nel primo plot sono le varianze previste per un numero di layer infinito.

L'andamento in funzione di p , invece, non rispetta le predizioni teoriche. Ci aspettavamo una crescita come p^2 per il fast entangling e come $p/(2-p)$ per lo slow, invece osserviamo due andamenti decrescenti.

La causa di questa discrepanza è, probabilmente, la scelta di un sistema con un numero di bit piccolo, che non evidenzia fenomeni di assorbimento. Questa scelta imprecisa è stata però obbligata da necessità computazionali, in quanto il costo di calcolo aumenta esponenzialmente con n .

Altri Circuiti

Le stesse considerazioni possono essere estese ad altri circuiti.

In particolare vediamo:

```
In [13]: def generate_random_parameters(n):
          return 2 * np.pi * np.random.rand(2 * n)
          # Numero di qubit
n=4
L_final=5
noise_gate = Gate(name="Noise", num_qubits=n, params=[])
circuit = QuantumCircuit(n)
rho = DensityMatrix.from_instruction(circuit).data # Matrice densi

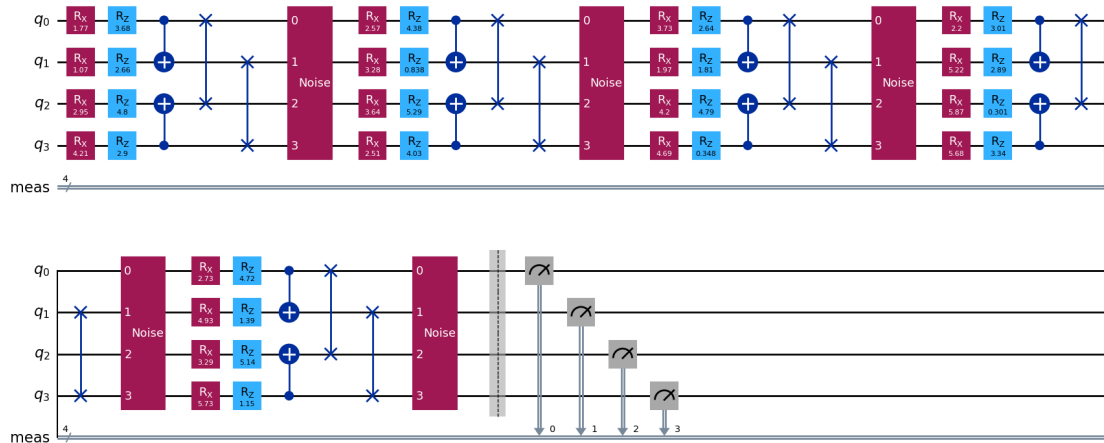
for _ in range(L_final):
    # Applica rotazioni casuali
    parameters = generate_random_parameters(n)

    for i in range(n):
        circuit.rx(parameters[2 * i], i)
        circuit.rz(parameters[2 * i + 1], i)
    circuit.cx(0,1)
    circuit.cx(3,2)
    circuit.swap(0,2)
    circuit.swap(3,1)

    circuit.append(noise_gate, range(n))

circuit.measure_all()
circuit.draw('mpl')
```

Out[13]:



```
In [14]: from inspect import trace
import numpy as np
import itertools
import matplotlib.pyplot as plt
from collections import defaultdict
from scipy.sparse.csgraph import connected_components
import qiskit as qc
from qiskit_aer import Aer
import numpy as np
from qiskit import QuantumCircuit
from qiskit.quantum_info import DensityMatrix, SparsePauliOp
```

```

n = 4 # Numero di qubit

def pauli_matrices_divided():
    sqrt2 = np.sqrt(2)
    return {
        'I': np.array([[1, 0], [0, 1]], dtype=complex) / sqrt2,
        'X': np.array([[0, 1], [1, 0]], dtype=complex) / sqrt2,
        'Y': np.array([[0, -1j], [1j, 0]], dtype=complex) / sqrt2,
        'Z': np.array([[1, 0], [0, -1]], dtype=complex) / sqrt2
    }

# Creazione delle matrici di Pauli divise
pauli_matrices = pauli_matrices_divided()

# Definizione delle etichette delle matrici di Pauli
matrix_labels = list(pauli_matrices.keys()) # ['I', 'X', 'Y', 'Z']

# 2. Genera tutti i possibili prodotti tensore e definisce la "binary tuple"
tensor_products = []
for combination in itertools.product(matrix_labels, repeat=n):
    tensor = pauli_matrices[combination[0]]
    for label in combination[1:]:
        tensor = np.kron(tensor, pauli_matrices[label])
    binary_tuple = tuple(1 if label != 'I' else 0 for label in combination)
    tensor_products.append((binary_tuple, tensor))

# Ordinamento in base al valore della tuple (facoltativo)
tensor_products.sort(key=lambda item: sum(bit * (2**i) for i, bit in enumerate(item[0])))

# Raggruppa i tensori in base alla binary tuple
groups = defaultdict(list)
for binary_tuple, tensor in tensor_products:
    groups[binary_tuple].append(tensor)

# Lista ordinata delle tuple uniche e definizione del numero J di gate
unique_tuples = sorted(groups.keys(), key=lambda tup: sum(bit * (2**i) for i, bit in enumerate(tup)))
J = len(unique_tuples)

#####_-----
# 3. Definizione della matrice H (in questo caso 4 gate pauli X)
n = 4 # Numero di qubit
circuit = QuantumCircuit(n)

circuit.cx(0,1)
circuit.cx(3,2)
circuit.swap(0,2)
circuit.swap(3,1)

H = np.array(Operator(circuit))

T = np.zeros((J, J), dtype=complex)

```

```

for j_idx, tup_j in enumerate(unique_tuples):
    LT_vec = np.zeros(J, dtype=complex)
    # Calcola il fattore di normalizzazione per il gruppo B: per ogni gruppo B
    norm_factor_B = 3 ** (sum(tup_j))

    # Per ogni tensore B nel gruppo j
    for B in groups[tup_j]:
        # Ciclo sui gruppi per P (indice k)
        for k_idx, tup_k in enumerate(unique_tuples):
            trace_sum = 0
            # Fattore di normalizzazione per il gruppo P
            norm_factor_P = 3 ** (sum(tup_k))
            for P in groups[tup_k]:
                trace_val = np.trace(H.conj().T @ B @ H @ P)
                # Normalizzazione: il contributo viene diviso per i fattori di normalizzazione
                trace_sum += (trace_val**2) / (norm_factor_B * norm_factor_P)
            LT_vec[k_idx] += trace_sum
    T[j_idx, :] = LT_vec

# 5. Diagonalizzazione a blocchi fortemente connessi tramite permutazione
# Costruiamo una "matrice di connettività": consideriamo un arco tra due gruppi se esiste un tensore che li collega
tol = 1e-16
connectivity = (np.abs(T) > tol).astype(int)

# Trova i componenti connessi (trattando la matrice come grafo non orientato)
n_components, comp_labels = connected_components(connectivity, directed=False)

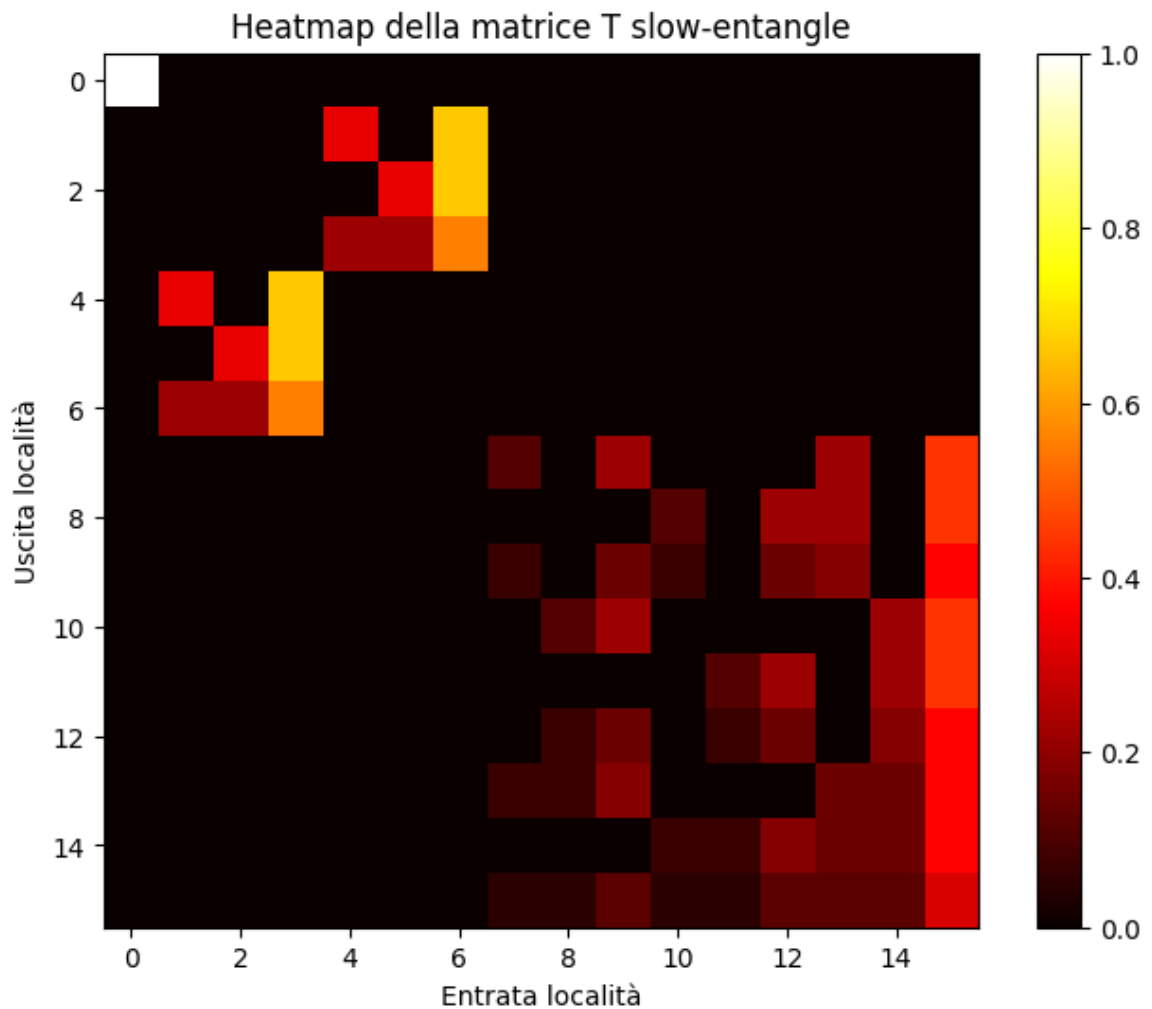
# La permutazione che raggruppa per componente è data da:
permutation = np.argsort(comp_labels)
# Riordiniamo T per ottenere una forma bloc-diagonale
T_perm = T[permutation, :][:, permutation]

# Salviamo la mappatura della permutazione: (nuovo indice -> indice originale)
perm_mapping = {new_idx: old_idx for new_idx, old_idx in enumerate(permutation)}

# 6. Plot della matrice T permutata come heatmap (si plottano i valori reali)
plt.figure(figsize=(8, 6))
plt.imshow(np.real(T_perm), cmap='hot', interpolation='nearest')
plt.colorbar()
plt.xlabel("Entrata località")
plt.ylabel("Uscita località")
plt.title("Heatmap della matrice T slow-entangle")
plt.show()

# Salva (o stampa) la mappatura della permutazione
print("Mappatura della permutazione (nuovo indice -> indice originale)")
print(perm_mapping)

```



Mappatura della permutazione (nuovo indice $\rightarrow$ indice originale):
 {0: 0, 1: 1, 2: 2, 3: 3, 4: 4, 5: 8, 6: 12, 7: 5, 8: 6, 9: 7, 10: 9, 11: 10, 12: 11, 13: 13, 14: 14, 15: 15}

I due blocchi potrebbero non essere diagonali per via di elementi infinitesimi che non riusciamo a vedere. Questo circuito sembra introdurre un entangle forte, ma non così forte da creare un unico blocco irriducibile. Tuttavia, ricordiamo che la dimensione ridotta considerata può essere un fattore importante in questa considerazione, infatti abbiamo una LTM molto simile a quella della cascata di CCNOT.

```
In [15]: from inspect import trace
import numpy as np
import itertools
import matplotlib.pyplot as plt
from collections import defaultdict
from scipy.sparse.csgraph import connected_components
import qiskit as qc
from qiskit_aer import Aer
import numpy as np
from qiskit import QuantumCircuit
from qiskit.quantum_info import DensityMatrix, SparsePauliOp, Operator

# Parametri
n=4
```

```

shots = 1000 # Numero di esecuzioni
L_values_range = 25 # Numero massimo di blocchi di operazioni
p = 0.1 # Probabilità di applicare lo stato GHZ
h = 1 # Coefficiente nell'Hamiltoniano

# Funzione per generare parametri casuali
def generate_random_parameters(n):
    return 2 * np.pi * np.random.rand(2 * n)

# Funzione per mescolare rho con lo stato GHZ

def rho_noise(p, rho):
    rho_ghz = np.zeros_like(rho)
    rho_ghz[0, 0] = 0.5
    rho_ghz[-1, -1] = 0.5
    rho_ghz[0, -1] = 0.5
    rho_ghz[-1, 0] = 0.5
    return (1-p)* rho + (p) * rho_ghz

# Funzione per costruire il circuito e applicare il rumore
def circuit_with_noise(n, p, L):
    circuit = QuantumCircuit(n)
    rho = DensityMatrix.from_instruction(circuit).data # Matrice d
    # Applica rotazioni casuali
    parameters = generate_random_parameters(n)
    for i in range(n):
        circuit.rx(parameters[2 * i], i)
        circuit.rz(parameters[2 * i + 1], i)
    circuit.cx(0,1)
    circuit.cx(3,2)
    circuit.swap(0,2)
    circuit.swap(3,1)

    W=np.array(qc.quantum_info.Operator(circuit))
    for _ in range(L):
        # Evoluzione dello stato
        rho = W @ rho @ W.conj().T
        rho = rho_noise(p, rho)
    return rho

    W=np.array(qc.quantum_info.Operator(circuit))
    for _ in range(L):# Evoluzione dello stato
        rho1 = W @ rho1 @ W.conj().T
        rho1 = rho_noise(p, rho1)
    return rho1

# Creazione dell'Hamiltoniano W
pauli_strings = []
coefficients = []

for k in range(n-1):
    # Coefficiente del termine
    coefficients.append(h *(2*(n/2)))
    # Stringa di Pauli per Z_k ⊗ Z_{k+1}
    pauli_string = ['I'] * n
    pauli_string[k] = 'Z' # Z_k

```

```

    pauli_string[(k + 1)] = 'Z' #  $Z_{\{k+1\}}$  (periodico)
    pauli_strings.append(''.join(pauli_string))

# Creazione dell'Hamiltoniano come SparsePauliOp
W = SparsePauliOp(pauli_strings, coeffs=np.array(coefficients))

# Lista per memorizzare i valori di L
L_values_all = {L: [] for L in range(1, L_values_range + 1)}

for L in range(1, L_values_range + 1):
    L_values_shots = []

    for _ in range(shots):
        rho = circuit_with_noise(n, p, L) # Stato con rumore
        W_matrix = W.to_matrix()
        L_value = np.trace(W_matrix @ rho.data) # Loss
        L_values_shots.append(L_value)

        rho1 = circuit_with_noise_slow(n, p, L) # Stato con rumore
        W_matrix = W.to_matrix()

    L_values_all[L] = L_values_shots

L_means = {L: np.mean(L_values_all[L]) for L in range(1, L_values_range + 1)}
L_variances = {L: np.var(L_values_all[L]) for L in range(1, L_values_range + 1)}

# Plot della varianza in funzione di L
L_range = list(L_variances.keys())
var_values = list(L_variances.values())
L_values = list(L_means.values())

plt.figure(figsize=(10, 5))
plt.plot(L_range, var_values, marker='o', linestyle='--', color='r')
plt.xlabel('L (Numero di blocchi di operazioni)')
plt.ylabel('Varianza della Loss')
plt.title('Varianza della Loss in funzione di L')
plt.axhline(y=10*5.33*p**2, color='r', linestyle='--', label="Linea di riferimento")
plt.grid(True)
plt.show()

var_values_array = np.array(list(L_variances.values()), dtype=float)
# Sottrazione corretta
var_values_log = var_values_array - (10 * 5.33 * p**2)

p_values = np.linspace(0.05, 0.5, 20)

for p in p_values:
    L_values_shots = []
    for _ in range(shots):
        rho = circuit_with_noise(n, p, L_values_range) # Stato con rumore
        W_matrix = W.to_matrix()
        L_value = np.trace(W_matrix @ rho.data)
        L_values_shots.append(L_value)
    L_values_all[p] = L_values_shots

L_means = {p: np.mean(L_values_all[p]) for p in p_values}

```

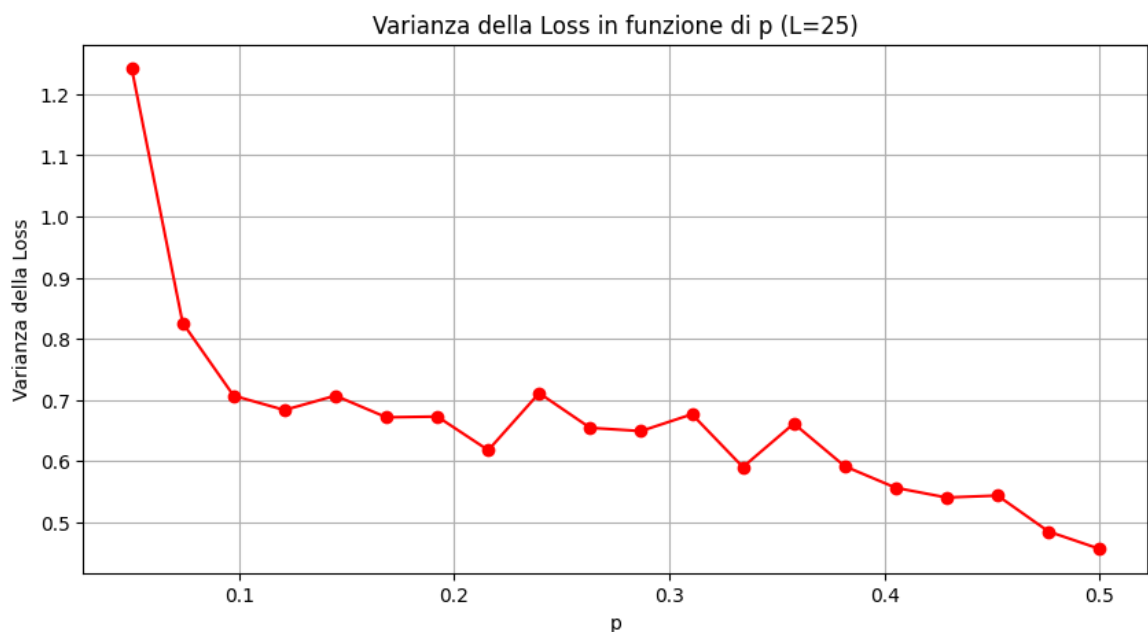
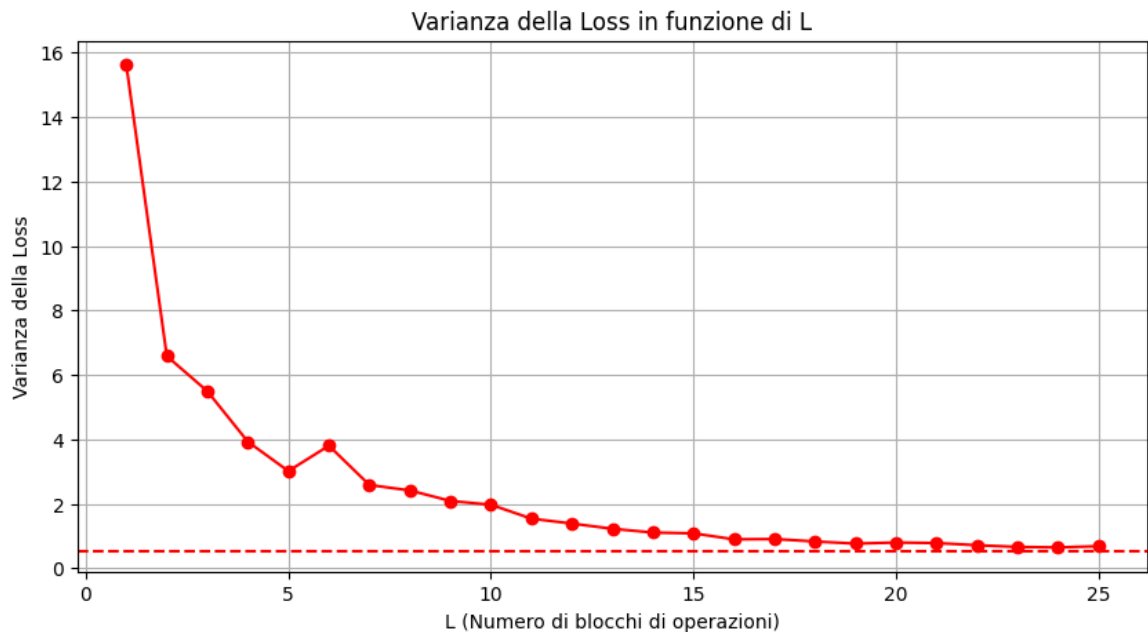
```

L_variances = {p: np.var(L_values_all[p]) for p in p_values}

var_values = list(L_variances.values())
L_values = list(L_means.values())

# Plot della varianza in funzione di p
plt.figure(figsize=(10, 5))
plt.plot(p_values, var_values, marker='o', linestyle='-', color='r')
plt.xlabel('p')
plt.ylabel('Varianza della Loss')
plt.title(f'Varianza della Loss in funzione di p (L={L_values_range}')
plt.grid(True)
plt.show()

```



L'andamento in funzione di L è quello atteso.

Per quello in funzione di p, valgono considerazioni analoghe agli esempi precedenti.

